

Major Exam

● Graded

Student

Sarthak Panda

Total Points

22.25 / 60 pts

Question 1

Q1

7 / 10 pts

1.1

A

5 / 5 pts

✓ + 2.5 pts *sigret* is responsible for invoking the kernel to restore the user thread's context

✓ + 2.5 pts *ret* pops return address from the stack and jumps to it. It does not restore the context, hence if we replace *sigret* with *ret*, the execution resumes with corrupted state.

+ 0 pts Incorrect

1.2

B

2 / 5 pts

+ 3 pts Stack overflow can overwrite return addresses, and when signal handler returns, it can open way for attacks.

✓ + 2 pts A different stack is used for handling signals to make sure that no program gets illicit access to the kernel as kernel level functions can be compromised.

+ 1 pt Partial explanation of stack overflow

+ 1 pt Partial explanation why different stack used

+ 0 pts incorrect

Question 2

Q2

1.5 / 10 pts

2.1 — A

1 / 4 pts

✓ + 1 pt Yes
secret() can be invoked

+ 3 pts Correct Explanation:
buffer[10] overwrites the return address of foo()

✓ + 0 pts Incorrect Explanation

+ 0 pts Incorrect/Blank.

2.2 — B

Resolved 0.5 / 2 pts

+ 2 pts Correct.

✓ + 0.5 pts No, secret won't be invoked.

+ 1.5 pts Heap grows upwards, while stack grows downwards. There is a huge gap between stack and heap. Stack stores the return address of foo. **Buffer overflow in heap won't overwrite the return address of foo in the stack. Thus secret won't be invoked when foo is returns back to main.**

✓ + 0 pts Incorrect.

+ 1 pt Exception: There is almost negligible probability secret being invoked when heap overwrite will update the return address of foo with start address of secret. It will happen only if stack and heap have grown very much, and their top are almost touching.

🔄 Regrade Request

Submitted on: May 11

Should not i get partial mark for "No, secret won't be invoked"?,

ok.

Reviewed on: May 12

2.3 — C

0 / 4 pts

+ 1 pt Identified stack overflow attack in the given context.

+ 3 pts Correct: Use of Canary Tokens

A random value stored between local variable (buffer) and the return address. If the buffer overflows, the random value is overwritten & detected later.

✓ + 0 pts Incorrect/Blank

Question 3

Q3

Resolved 4 / 10 pts

✓ + 0 pts Incorrect/Unattempted

+ 2 pts Top 32 bits are used to store timestamp

+ 2 pts `update()` along with placing the values also increments the timestamp

+ 4 pts `snapshot()` reads the timestamps twice, and if unchanged it returns else it tries again

+ 2 pts Justification of global progress (At least one thread completes)

💬 + 4 pts Point adjustment

🔄 Regrade Request

Submitted on: May 11

Sir, why my approach is marked as incorrect without any justification. My approach was simple and upper 32 bits were used to store the latest commit/update to location/array index by write[Update] operation. If the latest reader[Snapshot] (reader that came last crosses that position/index which has pending commit/update it writes back to lower 32-bits of array[LA]). This approach though different from the timestamp approach but I believe it works. My approach ensures all snapshots finish in finite steps [on avg. $O(n)$] and hence is lock-free, and all updates are in $O(1)$ because they write in UA, which is later committed to LA by latest reader. Please consider looking into my approach, I have written this idea in pseudo code form.

Giving partial marks for lock-free algorithm.

Reviewed on: May 12

Question 4

Q4

8 / 10 pts

4.1

A

3 / 3 pts

✓ + 1.5 pts High overheads

Sending an IPI to all cores forces each core to context switch and handle the interrupt, increasing CPU usage and latency

✓ + 1.5 pts Grace period

Determining when grace period ends require acknowledgement from all the other cores, which adds complexity and synchronization overhead, making the process expensive.

+ 0 pts wrong/Not Attempted/Unrelated

4.2

B

5 / 7 pts

+ 0 pts Click here to replace this description.

✓ + 2 pts **Design:** Tree RCU organizes per-CPU quiescent state data into a tree structure.

Each leaf node corresponds to a CPU's rcu_data structure.

Internal nodes are rcu_node structures that aggregate quiescent state info from child nodes.

+ 2 pts **Benefits:** Parallel Aggregation: Each internal node summarizes the state of its children. Only the root node needs to be checked to determine if all CPUs have entered the quiescent state.

This structure supports parallel updates and hierarchical checking, which avoids bottlenecks and makes the system highly scalable.

+ 5 pts Detection of the end of the Grace Period efficiently by checking bit when following condition met:

-> When a thread blocks (you know it is out of the read lock-unlock phase)

->Switches to user-mode execution

->Enters an idle loop

->Switch the context to run a kernel thread

+ 1 pt partial explanation of design

✓ + 1 pt Partial mention of benefits

✓ + 2 pts Partial mention of how grace period ends

Question 5

Q5

1 / 10 pts

5.1 — A

0 / 3 pts

+ 3 pts exact formulation $\text{Virtual_Address} = \text{vma.start_address} + i \ll 12$, and explained the parameters of the expression

+ 0.5 pts If linear search explained

✓ + 0 pts Incorrect

+ 2.5 pts correct formulation with explanation but multiplying index by page size missing

+ 1.5 pts Partial concept explained but formulation missing

+ 1 pt Some introductory concept

+ 0 pts [Click here to replace this description.](#)

+ 2 pts formulation written but explanation lacking

5.2 — B

0 / 3 pts

+ 3 pts If forking ,CoW, many to one mapping between vmas to anon_vma and after child page modification, one to many mapping between vma to anon_vmas explained

+ 1.5 pts Partial concept explained

✓ + 0 pts Incorrect

5.3 — C

1 / 4 pts

+ 1 pt Cache-behaviour:

For smaller anon_vma_chain sizes, sorted array would perform better due to better cache locality. However, for larger datasets or ones with freq modifications, the cost of array resizing and shifting of elements negate benefits

+ 1.5 pts Concurrency:

Red black tree more suited for anon_vma_chain operations because they are frequent and occur under contended locks. Tree rotations and node updates are localized allowing concurrent operations on different part of tree with minimum contention

+ 1.5 pts Efficiency

Red black tree more efficient for dynamic workloads with frequent insertions and modifications as seen in anon_vma_chain during page faults, fork, mem reclamation

+ 0 pts Unattempted/Wrong Answer

💬 + 1 pt Point adjustment

Question 6

Q6

0.75 / 10 pts

6.1 — A

0 / 2 pts

+ 2 pts when there are no pages left in the oldest generation.

✓ + 0 pts Incorrect

6.2 — B

Resolved 0.25 / 2 pts

+ 2 pts when no temporal locality exist

✓ + 0 pts Incorrect

+ 1 pt Partial. Explained in detail related to swappiness.

1. Swappiness != 0 && min_seq[lrugen_ANON] < min_seq[lre_gen_FILE]

2. 1 false and swappiness = 200

3. 1,2 false and File-.refault>=64 &&For file (NRR/ gain[file]) > for anon (NRR/gain[anon])

✓ + 0.25 pts partial about swappiness.

🔄 Regrade Request

Submitted on: May 11

Please sir look at Lecture -6 [Memory Systems] slide 81, there it is mentioned which file type to evicted there it is mentioned "Choose ANON if swappiness = 200", further my example also in align with the given example in rubrics, if there is more memory intensive tasks that means new data is fetched in every next step and hence no temporal locality exist, in contrast if it is a computational task it would have worked on small set of data to do some analysis and hence benefitting from temporal locality. Please consider looking into it.

Regraded.

Reviewed on: May 12

6.3 — C

Resolved 0.5 / 2 pts

+ 2 pts when there is access to large file then can evict older file pages

✓ + 0 pts Incorrect

+ 1 pt Partial marks if you write swappiness correctly

💬 + 0.5 pts Point adjustment

🔄 Regrade Request

Submitted on: May 11

Please sir look at Lecture -6 [Memory Systems] slide 81, there it is mentioned which file type to evicted there it is mentioned "Choose FILE if swappiness = 1", further in my example if computationally intensive task goes on then it is likely we are only working on small subset of data and doing analysis on it, in which case the old FILE pages which are not being currently used have less priority and can be evicted. Please consider to look into it. Thank you.

not completely correct

Reviewed on: May 12

+ 4 pts Prefetching/read-ahead mechanisms: The kernel proactively loads adjacent file data into the page cache, triggering accessed bits even if the application doesn't use them.

+ 2 pts Partially correct

✓ + 0 pts Incorrect

Operating Systems End Term Exam

2024-25 Sem II

Name:

Sarthak Randa

Entry number:

2022CS51217



COL 331



COL 633



ELL 405



ELL 783

Total: 60 marks

- Each question is for 10 marks.
- Prove and justify all your answers.

1. Answer the following questions:

- a. What happens if we replace the *sigreturn* system call with a *ret* instruction? [5]

Sigreturn is used to return from a signal handler. It helps to reset the context of process who invoked the signal and return to a address glibc wrapper who made a system call to invoke it. If we simply use a ret then the context (like registers, MSR etc) might be inconsistent.

- b. What is the advantage of using a different/alternate signal stack? [5]
 [Hint: Look at the next question (Question #2). There are security/correctness issues. Explain with examples and provide all the relevant details.]

Since the signals are handled in higher priority context than usual user process then using the same ^{user space} stack can pose security issues. Further if signal handler itself has exceptions then the user space stack can't be used from point where it left leading to correctness issues.

Eg:

```

void sighandler {
    int x = 5;
    ...
    return
}

int main {
    signal(SIG1, sighandler);
    ...
    kill(SIG1)
}
  
```

if signal invoked and same stack used user level process can see all the local variables & computer sig handler doing leading to security & correctness issues

→ register signal

→ invoke signal

2. Consider the following piece of code:
 void secret() {

```

...
}
void foo(){
    int j, buffer[5];
    for (j=0; j < 5; j++) buffer[j] = 14;
    buffer[10] = (int) (&secret);    /* Line BUF */
}

int main(){
    foo();
}

```

- a. Can the function `secret` get invoked because of the code in Line BUF? If yes, how? If not, why not? [4]

Yes it will get invoked in this case since `buffer` is on stack the nearby addresses to `buffer` still valid so when it tries to access the address of `secret` the function at the address will get executed.

- b. How will the answer to part (a) get modified if `buffer` is stored on the heap? [2]

NO, because buffer [10] would become an illegal/incorrect access since index 10 is out of bounds/size of array allocated. So the program will likely throw an exception.

- c. C does not perform "array bounds checking"; however, other languages such as Java do it. The reason is that every array access should not be burdened with an additional check operation. What is an efficient way of performing this check without burdening every access with additional instructions? The issue is that the array indexes are out of range. Consider an array of size 10. The user may try to access entries that are in the range 11-15, which is erroneous. Assume that such bugs are limited to accessing array indexes that are *slightly* out of range. They are not *severely* out of range. Propose a solution to detect such out-of-range accesses. They need not be detected immediately. They can be detected slightly later also. [4]

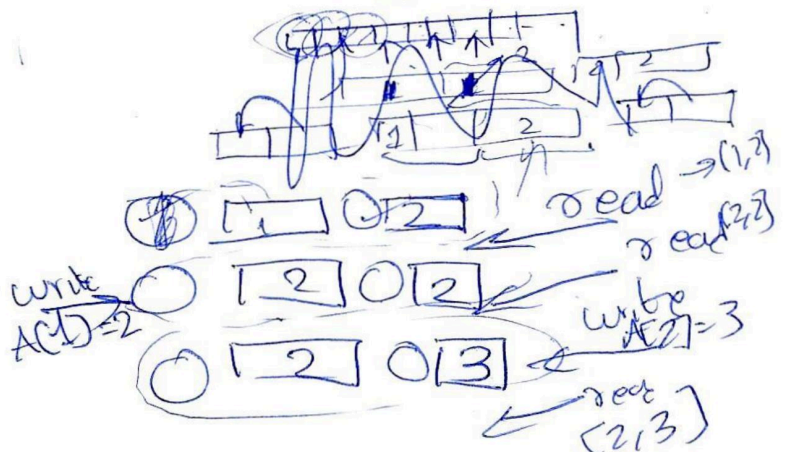


3. Consider an N -element array. We wish to support two concurrent operations:
 - a. Any thread can update any entry (one at a time). Consider the i^{th} entry.
Example: $A[i] = 3$
 - b. Collect an atomic snapshot of all the entries. This means that we need to collect the instantaneous state of the entire array that existed at a certain point in time.

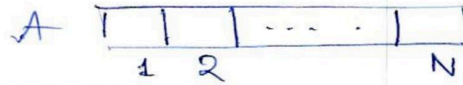
Assume that the array starts out to be $\{1, 2\}$. A process P is trying to collect a snapshot. It reads the first element as 1 and then waits for some time. The array is then modified to $\{2, 3\}$. P will then read the second element as 3, and will return $\{1, 3\}$ as the snapshot. The fact is that $\{1, 3\}$ was never the state of the array. It was either $\{1, 2\}$, $\{2, 2\}$ or $\{2, 3\}$.

How do we implement the *update* and *snapshot* operations using a lock-free program? This means that the entire system should make progress, i.e., one of these operations should always complete for some threads. Do not propose trivial or incorrect solutions.

Assume that every entry is a 64-bit integer: lower 32 bits are used to store the value, and the remaining 32 bits can be used for any other purpose. Thoroughly justify your answer. [10]



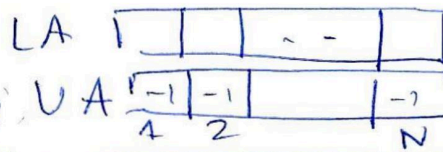
Let us assume array A .



~~to support~~ Assume that to take atomic snapshot of array A the reader reads from left to right then our ~~design~~ design should be such that we are able to keep the values of all the indexes to right unchanged until our snapshot is complete.

we can think of A as $LA \& UA$ for simplicity $LA[i] =$ value in lower 32-bits of $A[i]$
 $UA[i] =$ value in upper 32 bits of $A[i]$

$reader_idx$ ~~current~~ $\rightarrow$ current index reader i is reading.
 $MIN_reader_idx \rightarrow \min(reader_idx_i)$ at that moment.



Note we are not not handling overwrite by checking $UA[i] = -1$ since only need is snapshot consistency not latest.

Update min_reader_idx atomically using CAS.

Reader can directly read from LA array

If there are uncommitted writes in $UA[i]$ & min_reader_idx is crossing that index we can commit to LA array

~~UPDATE(A, i):~~ $UPDATE(A, i):$ If ($MIN_reader_idx == -1$)
 $LA[i] = V;$
~~ELSE~~ $UA[i] = V;$
 Snapshot (A, res): $readers++;$ $\rightarrow$ atomic increment using CAS is assumed
 for ($i = 1$ to N):
 ① $MIN_reader_idx =$ if ($reader == -1$) idx ; else $MIN(MIN_reader_idx, idx);$
 ② $res[i] = LA[i];$
 ③ if ($UA[i] != -1$ & $i > MIN_reader_idx$)
 $\{ LA[i] = UA[i];$
 $UA[i] = -1;$

4. Answer the following questions regarding RCU:

a. Why is it very expensive to send IPIs to all cores and then figure out when the grace period has ended? [3]

④ If ($i == N$ & $readers == 0$)
 $MIN_reader_idx = -1$
 if last index & no more reader
 set MIN_reader_idx to -1 .

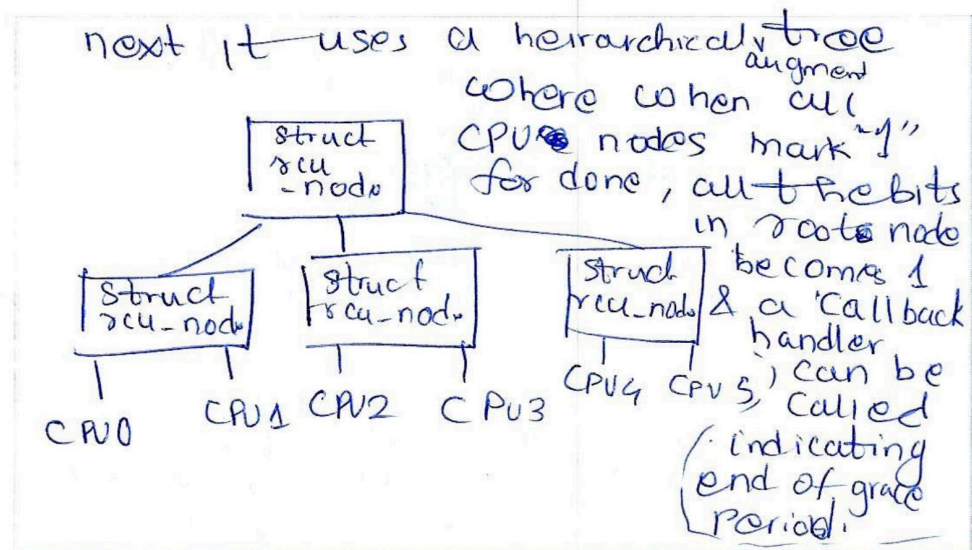
during a RCU ~~lock~~, the process becomes non preemptible on the CPU it is executing. Due to this the ~~general~~ ^{naive} idea to test RCU process has ended on other CPU to send IPI, if they are able to run successfully it means RCU unlock has been called & it is outside of RCU lock. Since IPIs themselves are costlier operation in term of time & resources, sending from multiple time to other CPUs for testing end of grace period is not very optimal.

- b. What is the more efficient mechanism that tree-RCU uses? How and why does it work? [7]

tree RCU uses the concept of Augmented tree ~~to track~~ RCU unlocks. It relies on event ^{Particular} (listed below) to track the RCU unlock has been called off or not:

- 1) Switching to another ~~process~~
- 2) ~~Invocation of~~ Interrupts

When these events occurs each CPU marks itself to be Quiescent state (lie outside of RCU read lock)



5. Explain the following regarding reverse mapping:
- Given a *struct page* and a *vma*, how do we find the virtual address in the *vma* that points to the page? [3]



- Justify the many-to-many mapping between *vm*s and *anon_vms*? [3]

- c. What will happen if the red-black tree is replaced with a sorted array? Consider cache behavior, concurrency and efficiency. [4]

In case red black tree is replaced with a sorted array then there is a possibility of false sharing, this was not the case of red black tree because they have fine tuned node size & branching factor so concurrent access of faraway ~~SPA~~ unrelated objects have less chance to fall in same cacheline which leads to false sharing. This is not the case with ~~red black tree~~ sorted array which effectively reduces its efficiency.

6. Answer the following questions in the context of the MGLRU algorithm.
a. When should *min_seq* be incremented? [2]

$$\frac{\# \text{ of Pages in min-seq generation}}{\text{MIN-NR-PAGES} + 2} < \frac{\# \text{ total Pages}}{\text{MIN-NR-PAGES} + 2}$$

↳ ie number of pages in min-seq are too less than threshold then we should increment the min-seq & age them/promote them.

b. When is it a good idea to evict ANON pages? Explain with examples. [2]

When the Swappiness hyperparameter is high it is good idea to evict ANON Pages. (ie Swappiness is 200 we are surely evicting ANON pages)

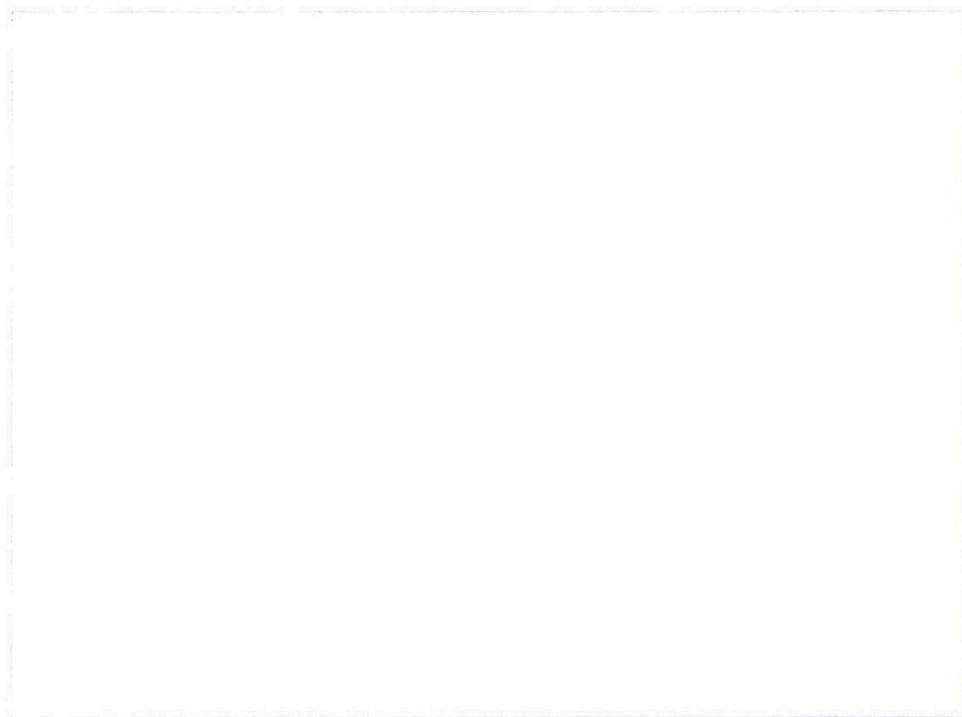
Eg: If nodes ~~do~~ do operations that involve memory related instruction than Computation then it is good idea to evict ANON pages

c. When is it a good idea to evict FILE pages? Explain with examples. [2]

When the Swappiness hyperparameter is low it is good idea to evict FILE Pages (ie if Swappiness is 0/1 we are surely evicting FILE pages)

Eg: If nodes ^{NUMA} do more computationally involved task which involves lesser memory access it is good idea to evict FILE Pages

d. Why could file-backed pages show artificially high refault rates, even if they have not been accessed? [Hint: Think about how the library and kernel code try to optimize file accesses.] [4]



----- Rough sheet -----

